Autonomous and Adaptive Cyber Incident Detection and Response in Industrial Cyber-Physical Systems Using Hierarchical Reinforcement Learning

AYESHA BABAR, School of Computing, Queen's University, Kingston, Ontario, Canada
TALAL HALABI, Universite Laval, Quebec, Quebec, Canada
MOHAMMAD ZULKERNINE, School of Computing, Queen's University, Kingston, Ontario, Canada

Cyber-Physical Systems (CPSs) are the backbone of many critical infrastructures. However, they have introduced an uncharted territory of security vulnerabilities and attack vectors, mainly due to the deeply integrated physical and cyber spaces. Moreover, in industrial CPS settings, network openness exposes the system to the outside world and renders it vulnerable to cyber threats. The security of industrial CPS significantly relies on the cyber incident detection and response systems which are fundamental to ensure the continuous and proper operation of cyber-physical processes. Among the key configuration parameters of these defense systems is the detection threshold. However, finding the optimal threshold that strikes the right balance between missed detection and false-positive rates remains a challenging problem. In this article, we propose a novel approach that leverages a Hierarchical Reinforcement Learning (HRL) architecture to autonomously detect the dynamic instability in an industrial CPS network and respond by adapting the cyber incident detection and response threshold range to minimize the effects of possible incidents. We developed and tested four HRL algorithmic variants, each offering potential avenues for optimization with its own strengths and limitations. Our agents dynamically select these ranges by assessing the expected risk and potential damage over time. In addition, the agent's selection process aims to minimize false positives and reduce the cost associated with changing the selected range. All four algorithmic adaptations show the effectiveness of HRL for designing adaptive cyber-physical defense compared to static approaches. Our experimental results indicate that our proposed technique is effective for building autonomous cyber incident detection systems in industrial CPS.

CCS Concepts: • **Computing methodologies** → **Reinforcement learning**; • **Networks** → *Cyber-physical networks*; *Network security*;

Additional Key Words and Phrases: Industrial CPS Security, Cyber Incident Detection and Response, Autonomous Cybersecurity, Adaptive Cyber Defense, Hierarchical Reinforcement Learning

This research is partially supported by the Natural Sciences and Engineering Research Council of Canada (NSERC).
Authors' Contact Information: Ayesha Babar (corresponding author), School of Computing, Queen's University, Kingston, Ontario, Canada; e-mail: ayesha.babar@queensu.ca; Talal Halabi, Universite Laval, Quebec, Quebec, Canada; e-mail: talal.halabi@ift.ulaval.ca; Mohammad Zulkernine, School of Computing, Queen's University, Kingston, Ontario, Canada; e-mail: mz@queensu.ca.

This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).
ACM 2378-9638/2026/1-ART8
https://doi.org/10.1145/3765622

ACM Reference format:
Ayesha Babar, Talal Halabi, and Mohammad Zulkernine. 2026. Autonomous and Adaptive Cyber Incident Detection and Response in Industrial Cyber-Physical Systems Using Hierarchical Reinforcement Learning. *ACM Trans. Cyber-Phys. Syst.* 10, 1, Article 8 (January 2026), 27 pages.
https://doi.org/10.1145/3765622

1 Introduction

Cyber-Physical Systems (CPSs) are complex, softwarized systems that encompass both cyber and physical components, making them vulnerable to various cyber intrusions and malicious activities that can cause significant physical damage. The recent advancements in AI have made cyber threats more complex and increasingly evolving. Indeed, many zero-day threats (exploiting yet an unexposed vulnerability) now rely on AI algorithms [48], making them even more unpredictable than before [24]. **Advanced Persistent Threats (APTs)**—sophisticated, multi-staged attacks that use various attack vectors [36]—have also created new security challenges for CPS in critical infrastructure due to their stealthy, dynamic, and adaptive natures and are hard to detect even with contemporary defensive measures [32]. A well-designed cyber incident detection system must address these threats through robust security measures to safeguard critical infrastructure and industrial processes including energy production systems, healthcare systems, and transportation systems [14]. However, current cyber incident detection and response strategies in CPS expose several limitations such as the high cost of implementing redundant systems, the computational efficiency required for real-time protection, and the challenges in securing the widespread and diverse nature of both IT and **Operational Technology (OT)** networks. Signature-based **Intrusion Detection Systems (IDSs)** are especially ineffective in detecting zero-day attacks [12]. Moreover, zero-day attacks can be present in a compromised system days before the security team can even detect them—62% of such attacks are detected after the system is compromised [43].

The limitations of incident response systems suggest a critical need for cybersecurity mechanisms that can adapt to rapidly evolving threats more efficiently. This may involve the autonomous update of threat detection systems, as it is often impractical or impossible to train humans quickly enough to perform this update in real time or under attack. AI has revolutionized cybersecurity with the help of advanced and autonomous systems, which promise a paradigm shift from manual, rule-based processes to self-learning, adaptive defenses. These AI capabilities allow for a proactive and efficient approach to cybersecurity, offering significant benefits in detecting and mitigating cyber threats autonomously. At the heart of this transition is **Reinforcement Learning (RL)**, a potent branch of machine learning that empowers systems to learn optimal actions through trial-and-error interactions with their environment. RL can significantly address the limitations of traditional defense techniques in network security by providing adaptive, dynamic, and proactive defense. An RL agent is normally provided with all the information about the environment via *observations* and *states*, and is able to interact with the environment by executing specific *actions*. The environment then provides feedback to the agent in the form of *Reward* based on how good or bad the actions are. Each agent has a strategy called *policy* from which it draws its actions, and the goal of any policy is to maximize rewards over time. Thus, RL methods can enable critical systems to learn and improve their defense strategies over time through interactions with the environment, which is particularly useful against evolving, AI-enabled cyber threats [33].

For industrial CPS, a cyber incident detection and response system normally monitors network traffic and system activities at both the cyber (IT network) and physical (OT network) layers in order to identify malicious actions or security policy violations. Central to the operation of any cyber

incident detection system are the threshold settings or the benchmark limits that play a crucial role in threat detection and prevention. These settings refer to predefined values or sets of criteria used to determine whether the observed network or system activities should be classified as normal or potentially malicious. An incident is detected if the monitored activity exceeds this threshold. The selection of an appropriate threshold is crucial, as it directly impacts the system's sensitivity and accuracy in detecting intrusions [38]. Traditional IDS may yield **False Positives (FPs)** for normal system behavior or missed detections depending on the configured threshold, which is often set based on static conditions or the current state of the system. An over-sensitive threshold can lead to issues like increased false alarms, while an under-sensitive threshold can result in missed detection. Setting an optimal threshold is crucial, as both FPs and missed detection, can cause disruptions in the overall CPS, leading to damages such as increased costs and downtime. However, industrial CPS are highly dynamic and normally engender risk factors that can change over time, which underscores the need for adaptive and optimized detection methods to maintain effective intrusion detection performance. Hence, industrial CPS networks require innovative security solutions that can strike a good balance between security level and system performance [36]. In this research, we propose and design an autonomous RL agent that dynamically adapts the cyber incident detection settings based on the current estimated risk by selecting predefined detection ranges. We seek to address the following two fundamental research questions: (1) How to optimally adapt the cyber threat detection threshold settings based on dynamic risk factors in an industrial CPS network and maintain the system's resilience against an active adversary? and (2) How to train an autonomous agent to effectively learn a multi-objective, optimal policy for controlling the IDS threshold settings solely via interacting with the CPS environment?

Nonetheless, replicating the complex threat surface of industrial CPS networks inside a self-learning environment with the goal of designing adaptive defense requires an effective approach for a long horizon planning. To address the problems where the agent needs to make a sequence of decisions over a large number of timesteps to achieve a goal, techniques like **Hierarchical Reinforcement Learning (HRL)** have proven very useful [28]. HRL divides the difficult task goals into easily achievable subgoals, providing an effective means to solve long-horizon planning tasks. It trains two levels of policies: (1) a high-level policy that decomposes the long-horizon tasks into simple short-horizon tasks, and (2) a low-level policy that focuses on these short-horizon tasks [31]. The subtask may itself be an RL problem with a lower-level policy [29]. The collective function of these hierarchical policies shapes the agent's behavior. By decomposing the original task into a series of subtasks, the long-term horizon of the initial task is shortened. This temporal abstraction results in each subtask acting as a higher-level action that endures over an extended timescale compared to lower-level actions [29]. Temporal abstraction can also enable efficient credit assignment over longer timescales [34]. At the same time, a subtask may itself be easier to learn, leading to a more structured exploration over the course of agent training [45]. The modularity inherent in HRL also allows for the potential reuse and transfer of learned sub-policies to related tasks or system configurations, thereby improving its ability to generalize [4].

In light of the identified limitations, our research makes the following significant contributions to the field of security in industrial CPS:

- We propose and formalize a novel approach to dynamically and autonomously derive the optimal threshold settings that balance between missed detection and FP rates for the cyber incident detection systems deployed within industrial CPS networks.
- We leverage the strengths of different variations of HRL algorithms to design the complex tasks of adaptive threat detection and mitigation in a dynamic CPS environment by breaking them down into manageable sub-tasks. The modeled HRL agents dynamically select

the threshold ranges by assessing the expected risk and potential damage on the CPS at any given moment. In addition, the agent's selection process aims to minimize FPs and reduce the cost associated with updating the selected range. To the best of our knowledge, this work is the first to incorporate HRL into designing autonomous cyber defense tasks in CPS.

— We implement two well-established HRL algorithms and evaluate their strengths and limitations for protecting an industrial control application. We compare against HQDN and PPO-based HRL since they represent the two dominant HRL paradigms: value-based (DQN) and policy-based (PPO) approaches. These foundational methods serve as solid baselines, and their comparison provides valuable insights into the trade-offs between these core HRL strategies.

— We evaluate our HRL agent's performance in a CPS setting by creating network traffic containing both IT and OT data. The data is calibrated using empirical values derived from a real-world network infrastructure and a review of diverse IoT datasets [3, 23, 58]. To enhance ecological validity, the evaluation incorporates time-varying intervals observed in real-time deployments, thereby approximating real-life operational scenarios as closely as feasible. The test environment is designed as an HRL problem and is used to evaluate two **Hierarchical Deep Q-Network (h-DQN)** and two Option-Critic agents. The results demonstrate that not only did the dynamic HRL agents effectively minimize the damage to the CPS, but also outperformed a static approach to deploying the IDS, which showcases the importance of autonomous and adaptive cyber incident detection and response in critical CPS applications.

The remainder of the article is structured as follows. Section 2 presents background information and the literature review. Section 3 introduces the preliminary concepts of HRL. Section 4 presents the proposed approach and describes the design and implementation of the HRL agents in detail. The results are discussed in Section 5. Finally, Section 6 concludes the article and presents potential future directions.

2 Background Information and Literature Review

This section provides a comparative analysis of traditional heuristic detection and response methods in contrast to RL-enabled solutions, then discusses the literature review related to autonomous cyber defense.

2.1 Autonomous Cyber Defense

With the rapid advancement of cyber threats, the digital landscape faces increasingly complex challenges that demand innovative, efficient, and proactive defense mechanisms. The rising frequency and sophistication of cyberattacks often exceed the capacity of human operators to respond effectively in real time. Timely decision-making is crucial, as delays in deploying appropriate defensive measures can lead to the compromise of critical assets. To address this limitation, there is a growing need for autonomous, adaptive, and resilient security systems capable of evolving in response to emerging and increasingly sophisticated adversarial strategies [17]. Recognizing the urgency of this issue, governments and national security agencies are prioritizing the development and deployment of autonomous defense systems to safeguard critical cyber-physical infrastructures. The U.S. **Department of Defense (DoD)**, for instance, has actively pursued the integration of AI into its defense architecture, including initiatives in autonomous network defense. The DoD's 2023 Data, Analytics, and Artificial Intelligence Adoption Strategy underscores the imperative to accelerate AI adoption in order to maintain decision superiority in network defense operations [18]. This strategic shift toward autonomous defense is expected to play a pivotal role in securing future

networked systems and protecting vital national infrastructure. Within this research, the concepts of autonomy and adaptivity are treated as related yet distinct. We define autonomy as the ability of a system or entity to make decisions independently, without external control or intervention. In contrast, adaptivity refers to a system's capability to adjust its behavior—specifically its parameters in our case—in response to dynamic environmental conditions [26].

2.2 RL for Cyber Security Tasks

Traditional IDS often rely on predefined rules or signatures to detect threats, which may not effectively counter new or evolving attacks. RL can help systems to learn and adapt to new threats without explicit reprogramming. For instance, **Deep Reinforcement Learning (DRL)** has been applied to enhance threat detection by integrating network security and endpoint security-based sub-systems into a more responsive and comprehensive detection and response system [51]. Also, RL can be trained to recognize subtle patterns that are indicative of APTs, which mostly engender sophisticated, long-term threats that traditional defense mechanisms struggle to detect due to their low and slow attack patterns [17, 49]. Furthermore, RL can automate the selection and implementation of defense strategies in real time, reducing the reliance on human intervention [35]. This is crucial for responding to attacks swiftly and efficiently to minimize potential damage. Since RL continuously learns from the environment, it can overcome the limitations of static defense mechanisms that do not evolve with the changing cyber threat landscape [16]. By constantly updating its knowledge base and strategies, RL-based defense can provide more robust protection against a wide range of cyber attacks [47] including zero-day vulnerabilities and sophisticated malware, improve the detection rate, and reduce FPs. The application of RL in cybersecurity heralds a new era of defense mechanisms capable of detecting, analyzing, and responding to threats in real time, significantly enhancing the resilience and efficiency of cybersecurity infrastructures [52]. In their comprehensive review of DRL methods in cyber security threat prevention and detection, Mohit et al. [51] convincingly demonstrate the potential of RL and DRL in transcending the inherent limitations of traditional rule-based and signature-based defense mechanisms, offering a promising avenue for the development of more resilient and adaptive cybersecurity approaches. Ali et al. [2] compared the latest AI approaches for zero-day attack detection and concluded that the RL approach achieved higher recall and F1-Score, which implies a better predictive ability (by examining performance on each class individually rather than considering the overall performance as is the case with the accuracy metric).

To solve the adaptive threshold problem in IDS, Ghafouri et al. [26] attempted to identify the optimal configuration or set of parameters that result in the lowest possible cost. This implies that there might be a single solution that balances between cost and performance. However, our cost calculation focuses on maintaining certain parameters within specific ranges of thresholds, potentially to meet particular CPS performance standards or constraints, e.g., related to quality or efficiency. We believe that our approach can lead to a wider range of acceptable solutions rather than a single optimal configuration and can be scalable if more factors that affect the cost are further involved. We also demonstrate that this is a better approach for designing the agent's reward function since it guides the agent in performing multiple action spaces rather than solely focusing on the cost, which makes it more fit for real-world CPS scenarios.

2.3 Dynamic CPS Environment Modeling for RL Tasks

For RL-based security task modeling, detection and response are infinite horizon problems, which means that the main objective is to find an approximately optimal policy for a system exposed to changing environmental parameters. Such systems do not follow the main assumptions made for the classical RL methods, and dynamic real-world complications make solving problems with

classical RL algorithms difficult. These algorithms are developed with basic underlying assumptions on the transition probability and reward functions. These assumptions are necessary, since RL algorithms are examples of stochastic approximation algorithms [13]. For an RL algorithm, to find the best solution (converge to the optimal value functions), there are two assumptions that need to be true [44]. First, the reward for any state-action pair must be limited and not infinitely large (i.e., is always within a certain range and never becomes infinitely positive or negative). This assumption is important for the mathematical tractability and stability of learning algorithms. Second, the dynamics of the environment in which the agent is operating including the system's transition probabilities and rewards must remain stable over time so the RL algorithm can learn effectively. This assumption simplifies the learning problem because the agent can learn a policy based on the assumption that the future will behave like the past. However, CPSs are dynamically varying environments where the second assumption does not hold. Since previously proposed RL algorithms [5, 11] are mainly suited for stationary environments, we need to develop new methods which autonomous agents can utilize to handle non-stationary environments. Following Padakandla et al. [44], we use the terms *dynamically varying environments* and *non-stationary environments* interchangeably and aim to tackle the challenge of controlling dynamic environments in the cyber-physical domain by proposing HRL-based solutions to control the cyber incident detection benchmark settings in an adaptive fashion.

2.4 HRL vs. RL: Theoretical and Empirical Comparisons

To solve the adaptive IDS threshold problem using RL, we propose incorporating the HRL abstraction into our reward and state representation. This will encourage the agent to learn the relationships between the high-level and low-level rewards. HRL integrates the principles of intrinsic motivation by giving state information in distinct ways at different levels [8]. The changes in state transitions and reward functions create a sequence of **Markov Decision Processes (MDPs)**, each drawn from an unknown distribution called a *context* [1]. These non-stationary environments also require a change detection mechanism, which involves designing and analyzing algorithms to estimate in real time when an important event occurs and identify properties of the behavior after the change [19]. This is a key challenge for optimal control in non-stationary MDP. Such a control problem aims to maximize the long-term discounted reward when the transition dynamics or the reward function can change over time. When prior knowledge of change statistics is available, the standard Bayesian approach to this problem is to reformulate it as a **Partially Observable MDP (POMDP)** and solve it using approximate POMDP solvers, which are typically computationally demanding. Banerjee et al. [7] formulate the problem as quickest change detection and provide a set of tools for detecting a change in the distribution of a sequence of random variables. The **CUmulative SUM (CUSUM)** is an approach that extends sequential operations over time by cumulatively summing the deviations from a target value [15]. In this context, CUSUM is used to build upon the previous tests and analyze sequential data over time [15]—we indirectly use this concept by calculating the normal ranges of the threshold. For infinite horizon problems, the change detection-based approach and the *CUSUM* approach usually work, and we leverage this notion of change detection by incorporating the two hierarchies in our state representation. This approach better estimates the transition and reward functions for all contexts; it also overcomes the hard-to-learn sparse rewards due to complex state representation. Also, Li et al. [40] proposed a notion of a reference state in the regret analysis that helps understand the deviation in a system that has a non-stationary policy. We apply this concept to our normal threshold computations and utilize the idea of reference states to define the ranges for anomalous, normal, and high alert thresholds.

The empirical evidence and theoretical advantages of HRL strongly support our choice of using HRL over conventional RL methods for dynamic environments in the cyber-physical domain [9].

HRL effectively addresses the non-stationarity challenge through subgoal re-labeling and leads to demonstrably better performance on continuous control tasks in MuJoCo robotics [56]. This is possible since the subgoals are (1) defined by selective features, for instance, using HIerarchical Reinforcement learning with Off-policy correction (HIRO) [42]; or (2) replaced with achieved states, e.g., Hierarchical Actor-Critic [39]. In many studies, HRLs have demonstrated enhanced performance compared to traditional RL. Similarly, Option-Critic [6] achieves superior results in Atari games. H-DRLN [55] also demonstrates improved performance and sample efficiency in Minecraft. HRL's hierarchical structure facilitates efficient learning and skill reuse, and H-DRLN significantly reduces sample complexity through multi-skill distillation.

3 Preliminary Concepts of HRL

This section presents the general concepts of two prominent HRL algorithms: h-DQN and Option-Critic, and describes their key components, learning mechanisms, and suitability for solving HRL problems. In HRL, the highest-level policy generally chooses the sub-tasks of the main task as its actions. This policy is trained to perform the main task in terms of the sequence of its sub-tasks using the rewards obtained in the main task. At a lower level in the hierarchy, a sub-task chosen by the higher level is itself an RL problem. A lower-level policy learns to perform that sub-task using the internal rewards related to it (optionally, the main task reward may also be added) [46]. The complexity of cyber-physical environments, particularly in industrial settings, can make even seemingly simple tasks, such as cyber incident detection configuration, challenging. For instance, reducing both the potential system damage due to cyber threats and the rate of IDS FPs is crucial yet challenging to achieve simultaneously, making it difficult for an RL agent to learn this as a single task due to the vast state space and sparse rewards [31]. Furthermore, distinguishing between cyber intrusions at the level of the overall network and specific CPS components presents another aspect of complexity, suggesting that dividing these tasks into smaller, more manageable sub-tasks may be more effective.

3.1 Deep HRL

Learning goal-directed behavior in environments with sparse feedback is a major challenge for RL algorithms [37]. The primary difficulty arises due to insufficient exploration, resulting in an agent being unable to learn robust value functions. Intrinsically motivated agents can explore new behavior for its own sake rather than to directly solve problems. Such intrinsic behaviors could eventually help the agent solve the tasks posed by the environment, as proposed by Kulkarni et al. [37] in the h-DQN model. The model is based on a Deep HRL approach and consists of a two-level hierarchy of DQNs. In DQN, a convolutional neural network is used to approximate the Q-value functions [41], and the agent is trained to learn the optimal policy for decision making. A DQN receives the states as input and outputs the number of possible actions for that state. The hidden layers in the basic DQN depend on the tasks and their magnitude, but they are all fully connected, and help extract the features from the input state. The second main variation of a DQN is a Dueling DQN [57]. The approach separates the estimation of the state value and the advantage of each action by introducing two networks, one for the state value function and one for the state-dependent action advantage function. This helps generalize the learning across actions without imposing any change to the underlying RL algorithm [57]. Other DQN variations include Prioritized Experience Replay [50], Noisy Networks DQN [25], Distributional DQN [10], and Rainbow DQN [30]. While all of these are designed to improve some aspect of DQN learning such as stability [10], exploration [25], and efficiency [50], none of these variations help in decomposing a task or a single action into manageable hierarchical structures.

3.2 Hierarchical Deep Q-Learning Algorithms

To effectively address the selection of dynamic threshold settings in our cyber incident detection system for industrial CPS, we explore a range of h-DQN architectures that have proven suitable for addressing HRL challenges. The main architecture proposed by Kulkarni et al. [37] consists of a Meta-Controller which learns a policy over options (high-level actions), and a Controller which learns a policy over atomic actions (low-level actions). While planning with temporally extended actions is well understood, creating such abstractions autonomously from data has remained challenging. Bacon et al. [6] tackle this problem by proposing a new option-critic architecture capable of learning both the internal policies and the termination conditions of options, in tandem with the policy over options and without the need to provide any additional rewards or sub-goals [53]. Another HRL approach presented by Dietterich et al. [21] consists of the decomposition of the task by representing it as an MDP and breaking it down into a hierarchy of smaller, more manageable MDPs, similar to dividing a large task into smaller sub-tasks. The feudal network approach is another HRL approach to speed up RL. In feudal networks [20], learning happens simultaneously at multiple resolutions in space and time. Based on the architectures proposed in the above review, the HRL framework can be designed in various ways summarized as follows:

- —h-DQN approach, where a Meta-Controller learns a policy over high-level actions, and a Controller learns a policy over low-level actions [37].
- —Two or Multi-DQN approach where n separate DQNs are trained, one for each *n-level* actions. The higher-level DQN outputs a state which is then used as input for the lower-level DQN. In this setup, coordinating the learning between different networks is a big challenge [51].
- —Option-critic approach, where both the high-level and low-level policies are learned simultaneously. The high-level policy, also known as the option policy, learns to select among a set of low-level policies, also known as options. Each option, when activated, takes control of the agent's actions until it decides to terminate [6].
- —Decomposed approach, where the value function is decomposed into a hierarchy of smaller value functions, each corresponding to a sub-task in the overall task [21].

Among these approaches, the h-DQN and option-critic architectures are used as the methodological framework for the research. A detailed explanation of these techniques is presented in the subsequent sections.

3.3 h-DQN HRL

The h-DQN architecture is based on a *Controller* and *Meta-Controller* components, where the agent learns using intrinsically motivated behavior. Consider an MDP represented by states $s \in S$, actions $a \in A$, and transition function $T : (s, a) \rightarrow s'$. The agent observes a state s from the environment and takes an action a, which results in a new state s'. The reward is a function of s, defined as $F : (s) \rightarrow \mathbb{R}$. The goal of the agent is to maximize this function over a long period of time. A typical RL agent follows an ϵ-greedy approach for exploration, which works well for local exploration but may fail to provide an impetus for the agent to explore different areas of the state space. This limitation can be addressed using the notion of goals $g \in G$, which serve as intrinsic motivation for the agent. Here, the agent is able to focus on achieving sequences of goals in order to maximize the cumulative extrinsic reward. h-DQN uses the notion of options [53] to define policies π_g for each goal g. Here, the agent learns option policies simultaneously along with learning the optimal sequence of goals to follow. In order to learn each π_g, the agent also has a critic, which provides *intrinsic rewards* based on whether the agent is able to achieve its goals.

The agent uses a two staged hierarchies, controller and meta-controller. Let G be the set of all possible current goals. The agent first receives the state s_t and chooses a goal $g_t \in G$. The controller selects an action a_t based on the current state s_t and goal g_t. Until the goal is either accomplished or reaches the terminal state, the goal g_t is maintained over a series of timesteps. The internal critic evaluates whether the goal has been met and provides a corresponding reward $r_t(g)$ to the controller. The objective of the controller is to maximize the cumulative intrinsic reward R as follows:

$$R_t(g) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r_{t'}(g). \tag{1}$$

Similarly, the meta-controller aims to maximize the cumulative extrinsic reward formulated as:

$$F_t = \sum_{t'=t}^{\infty} \gamma^{t'-t} f_{t'}, \tag{2}$$

where f_t represents the reward signals received from the environment. The rewards function is temporally dependent on the sequential history of the goals. This also gives reward functions their dynamic nature that makes them better suited for the dynamic control problems. To implement the h-DQN architecture, a Deep Q-learning framework is employed for both the controller and the meta-controller. The Q-value function for the controller is defined as:

$$Q_1^*(s,a;g) = \max_{\pi_{a|g}} \mathbb{E}\left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r_{t'} \middle| s_t = s, a_t = a, g_t = g, \pi_{a|g}\right], \tag{3}$$

where g is the agent's goal in state s and $\pi_{a|g} = P(a|s,g)$ represents the action policy. Similarly, for the meta-controller, the Q-value function is given by:

$$Q_2^*(s,g) = \max_{\pi_g} \mathbb{E}\left[\sum_{t'=t}^{t+N} f_{t'} + \gamma \max_{g'} Q_2^*(s_{t+N}, g') \middle| s_t = s, g_t = g, \pi_g\right]. \tag{4}$$

Here, N represents the number of timesteps until the controller stops for the current goal. The variable g' is the agent's goal in state s_{t+N}, and $\pi_g = P(g|s)$ denotes the policy over goals. It is important to note that the transitions (s_t, g_t, f_t, s_{t+N}) generated by Q_2 operate on a slower time scale than the transitions $(s_t, a_t, g_t, r_t, s_{t+1})$ generated by Q_1. We approximate $Q^*(s,g) \approx Q(s,g;\theta)$ using a non-linear function approximator with parameters θ, referred to as a DQN. The learning in the h-DQN is done using stochastic gradient descent. The meta-controller experiences are collected once the controller terminates, but the experiences for the controller are collected at every timestep. New goals, g, are sampled with ϵ-greedily, with the exploration rate ϵ_2 decreasing over time. The controller selects actions for each goal using an epsilon-greedy policy with exploration rate $\epsilon_{1,g}$, which is adjusted based on the goal's success rate.

3.4 Option-Critic HRL

The option-critic architecture [6] is based on the framework of *option* [53]. Options are temporally extended actions. The framework tries to address the problem of extended actions for creating such abstractions autonomously from data. The proposed architecture uses two gradient decent theorems for options. The theorems enable the resulting architecture to learn internal policies and the termination conditions of options without the need to provide any additional rewards or sub-goals. This approach is flexible and efficient for both continuous and discrete environments. The framework was proposed to bridge the gap between the problem of discovering the options and learning of the options. In the option framework [53], where $\forall s \in S, \forall \omega \in \Omega : s \in I_\omega$,

a Markovian option $\omega \in \Omega$ is defined as a triple $(I_\omega, \pi_\omega, \beta_\omega)$. Here $I_\omega \subseteq S$ is an initiation set, π_ω is an intra-option policy, and $\beta_\omega : S \rightarrow [0, 1]$ is a termination function. This structure operates within a call-and-return option execution model, where an agent selects and executes an option ω, and waits for it to terminate. Once the selected option is executed and terminated properly the agent returns to select a new option. The option ω is selected according to the option policy π_ω. The agent then iteratively follows the intra-option policy $\pi_{\omega,\theta}$ until the termination condition $\beta_{\omega,\theta}$ is met.

The overall setup supports a stochastic gradient descent algorithm to learn options. By using a two-timescale approaches, the values are updated quickly while adjusting the intra-option policies and termination functions more gradually. This architecture is motivated by the actor-critic architectures [53]. The option policy module takes the current state (s_t) as input, and outputs an option. The options are temporally extended actions (a_t) available to the agent at time t. The option-critic ($Q\omega$) module then evaluates the values of different options based on the current state $s(t)$ and the selected option, and outputs a Q-value ($Q_\omega(S_t, O_t)$) which represents the expected future reward that the agent can anticipate by executing the chosen option in the current state. R_t is the immediate reward received at time t for executing action a_t. The option policy (π) learns to select options that are expected to lead to higher future rewards based on the feedback from Q_ω. This architecture allows the agent to learn a hierarchy of control [6].

4 Autonomous and Adaptive Cyber Incident Detection for Industrial CPS

This section provides a detailed description of the proposed cyber defense solution powered by HRL. The HRL framework is first formulated by defining the model parameters and structural components relevant to the CPS under consideration. This is followed by an in-depth discussion of the algorithmic variants employed to address the problem of autonomous and adaptive threshold setting within the CPS environment. These algorithmic strategies are designed to enable intelligent, context-aware decision-making for enhancing system resilience against cyber threats.

4.1 General HRL Model

RL is defined as the Machine Learning approach to solve the MDP by training an agent through interactions with the environment, which consists of a set of states S, a set of actions A, a transition function $P : S \times A \rightarrow (S \rightarrow [0, 1])$ and a reward function $r : S \times A \rightarrow R$. To find the optimal policy for an RL agent, a value function helps calculate the reward and is based on the utility of any state s in achieving the agent's overall objective. In HRL, the control is defined as a problem of finding a policy that maximizes expected future rewards. A collection of these policies can be hierarchically arranged with temporal dynamics for learning or planning within the framework of semi-MDP [54]. To formalize the problem on different hierarchies, we need to define the dynamics of the problem in terms of varying levels of abstraction. In our proposed model, we used two hierarchies, higher (HL) and lower (LL) for our actions $\forall a \in A$ and states $\forall s \in S$. Each level has its information, associated goals, decisions, and the corresponding outcome of the decisions. We associate the higher level of abstraction to define the higher level states, actions, and rewards. Whereas the lower level of abstraction can be used to represent the lower-level states, lower-level rewards, and lower-level actions. Let $S = [S_{HL}, S_{LL}]$ be a set of states, where the subscripts HL and LL refer to high-level and lower-level states, respectively. Similarly, let $A = [A_{HL}, A_{LL}]$ define the set of actions at both levels. The agent also receives high and low-level reward signals $R = R_{HL} + R_{LL}$ to guide its learning process.

The proposed architecture is depicted in Figure 1. We define the autonomous agent as the entity that can control and decide the threshold range of the cyber incident detection system in the industrial CPS without external control or instructions. Our agent is designed to achieve this goal in an adaptive fashion by dynamically adjusting the threshold ranges based on CPS risk

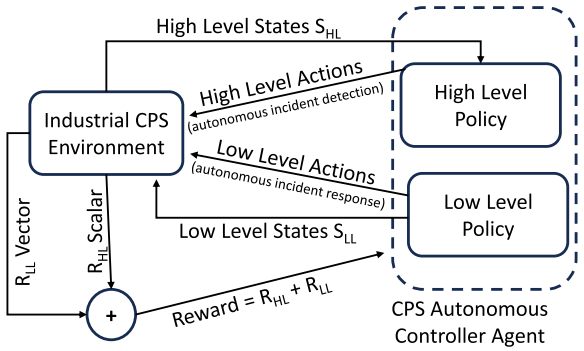

Fig. 1. The HRL-based CPS environment and agent modeling.

factors, as opposed to the static threshold approach. The agent operates within the industrial CPS environment—the virtual space where the autonomous agent executes the adaptive actions and receives feedback. The agent first monitors the high-level states and based on the information received performs a set of actions from the set of high-level actions. The results of these actions augment the state of the system for the agent with a more refined view of the environment. Subsequently, the agent is presented with related augmented information as a set of low-level states, from which it chooses to execute a low-level action. The agent receives the reward(s) for both actions from the environment, which then updates both high-level and low-level state representations. This iterative process continues throughout the task until the agent learns both the high-level and low-level policies. The distinction between states within the hierarchy can be implicit or explicit depending on the specific agent model. In some cases, the agent may receive a holistic state representation where the high-level and low-level abstractions are implicitly embedded within the input. Alternatively, the agent may be provided with separate, hierarchical state representations, making the distinction between levels explicit. Regardless of the representation, the high-level model and its interaction with the environment remain consistent. Regarding the reward structure, the high and low rewards signals can be employed in various configurations to influence the agents' learning trajectory. These signals may be used individually or in combination, depending on the specific learning process the agent adopts.

In the formulated HRL problem, the context involves selecting between different hierarchical levels within the network from a threat detection perspective. In our industrial CPS scenario, this might entail choosing between IT and OT network segments. The agent determines this selection as part of its high-level decision-making process. However, this hierarchical structure can be adapted to various network topologies and abstraction levels. On the other hand, the lower level helps the agent take action against the different involved factors, and the lower-level states provide the required details based on the context selected by the higher-level states (e.g., type of potential cyber intrusion). The agent identifies the context within a CPS environment and also decides on the lower-level actions without any external intervention or control. The lower level can further be fine-grained based on the abstraction and the actions required. The actions are also abstracted at two levels: At the high level, the actions consist in selecting the network or sensor context, while the actions at the lower level consist in selecting the appropriate ranges for the threshold based on the selected higher-level context. The total reward for the agent is the sum of rewards earned at both the high and low levels. We propose the high-level reward as a scalar value and the low-level reward as a vector representing the multi-objective reward according to various factors at that level.

Table 1. Option-Critic and h-DQN Comparison

Feature	h-DQN	Option-Critic
Sub-Task Definition	Explicit	Implicit
Learning Method	Q-Learning	Q-Learning and Policy Gradient
Flexibility	Low	High

We explore two HRL variations to solve the adaptive threshold problem and perform an empirical analysis. Both are used to break down complex tasks into simpler ones. We implement four HRL models: Two are based on the h-DQN implementation (explicit sub-tasks), and two are based on the option-critic implementation (implicit sub-tasks). h-DQN uses hierarchical Q-Learning and requires an explicit definition of sub-tasks. Option-critic utilizes the concept of options as temporally extended actions, and sub-tasks are learned directly from the environment. For the option-critic variation, we present all the information at once and the distinction between the high-level and low-level states is implicit in the states. The environment is updated in response to threshold modifications initiated by low-level actions. In h-DQN, both the high and low-level policies are learned using Q-networks, whereas option-critic learns both the sub-tasks and the policy for selecting them simultaneously, as summarized in Table 1. How each HRL agent interacts with the environment is different based on the algorithmic requirements. The specific implementation details of HRL can vary based on factors such as the method of feedback signal integration (combined or separate) and the information presentation strategy (simultaneous or staged). However, the fundamental principles of hierarchical structure, action selection, and reward assignment remain consistent across different HRL variations.

4.2 System Parameters

Figure 2 illustrates the interaction between our proposed IDS agent and a representative Industrial Control System, comprising two industrial pumps (P1 and P2) and two chemical chambers (C1 and C2) [27]. These components form part of a larger industrial CPS, which itself is embedded within a broader industrial control network that typically includes an enterprise layer and, in most cases, connectivity to the Internet. We assume the presence of a cyber incident detection and response system that continuously monitors network traffic and generates alerts upon identifying anomalies that deviate from predefined behavioral thresholds. Unlike static thresholds, the detection limits in our system are dynamically adjusted in response to changes in network conditions and operational context. The core of our approach is an autonomous, HRL-enabled threshold controller, which aims to maintain an optimal balance between FPs and missed detections. Striking this balance is critical: Excessive FPs can cause costly system downtime in industrial settings, while missed detections may lead to catastrophic failures or safety hazards. The agent is designed to autonomously select and adapt detection thresholds by evaluating the real-time risk level of the CPS. It continuously examines a range of system health indicators to make informed decisions that align with operational safety and performance requirements.

The industrial CPS scenario involves multiple dynamic variables, which are defined and discussed as follows:

— $C(a_{LL})$ denotes the cost of the currently selected low-level action a_{LL}, and is calculated against the low-level state once the context is selected by the agent.
— $D(s_{LL})$ denotes the expected damage due to the vulnerability in the current state, and is also calculated for the low-level states based on the context.

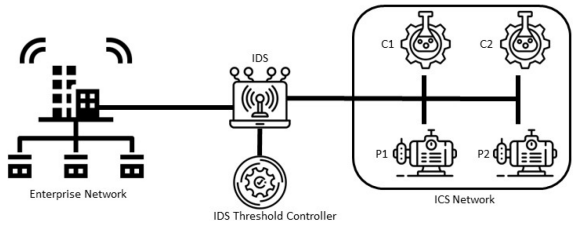

Fig. 2. The autonomous IDS agent's interactions with the industrial CPS environment.

- $D_k(t)$ denotes the detection flag for the kth parameter at time t. It is set based on the high-level actions.
- $FP_k(t)$ denotes the cost of FPs at time t for the kth parameter.
- $FN_k(t)$ denotes the cost of **False Negatives (FNs)** (missed detection) at time t for the kth parameter.
- η represents the current threshold and has two limits: the upper-bound and the lower-bound. It is also defined separately for both levels of the states. η will be used with respective superscript and subscript to properly represent the threshold. The superscript will be used to refer to the level of the states, and the subscript will be used to refer to the lower or upper bound of the threshold. For example, η_u^{kL} denotes the upper bound of the threshold of the current metric k on the lower-level state.
- α, β, γ, *and* δ represent weighting factors that modulate the influence of the respective low-level reward components. These weights can be tuned to prioritize specific aspects of the low-level performance, ensuring that the agent's overall behavior aligns with the desired objectives.
- The state space consists of scalar values representing high-level and low-level observations. At each timestep t, the agent receives a state vector s_t composed of high-level and low-level state variables. These observations serve as the basis for the agent's decision-making process and subsequent actions. For example, $s_H^k(t)$ refers to the scalar value of the parameter k at time t related to the high-level state.
- The current state vector s_t is used to compute deviation and detection scores at each timestep t.
- The detection flag, denoted as $D_k(t)$, is a binary indicator associated with the incoming state vector s_t. It signifies whether a cyber detection event has occurred at the current timestep.

4.3 State and Action Representations

The proposed hierarchical states decrease the complexity of information representation and make it less challenging for the agent to learn. The hierarchical approach also excludes irrelevant information from the state and is flexible enough to be easily extended to accommodate granular information. The approach helps the agent understand the granularity of the environment with the correct context and also provides more flexibility for the addition of newer context in the future. We refer to the high level as the network intrusion level, which provides all the observations related to the general security status of the network, represented as $s_H^k(t)$ (e.g., the time of the day, CPU usage, bandwidth). The low-level states are related to intrusions at the physical process level (e.g., sensors and actuators) and are represented as $s_L^k(t)$. Once the agent has identified the high-level information, it can focus on the information relevant to the current high-level state by querying the environment to extract the related details and augment the state with the most relevant information. This hierarchical and augmented state representation fine-tunes the agent's

understanding of the network to set the threshold. For example, a Denial of Service attack on the overall enterprise network (IT) will produce an anomaly in the bandwidth utilization or the packet exchange traffic ($s_H^k(t)$). On the other hand, a cyber attack targeting an industrial CPS component (OT) will present different symptoms, e.g., a sudden change in the flow ($s_L^k(t)$) is a better indication of such an attack on a pump and the number of packets might not be able to give a fine-tuned picture of the cyber incident. Similarly, we follow the hierarchical notion of abstraction of the action space and divide it into two levels: high-level actions and low-level actions. The high-level actions help the agent get the high-level reward to reinforce the context, while the low-level actions help the agent fine-tune the rewards for higher-level actions. The agent's decision-making process involves two hierarchical levels. At the strategic level, the agent can choose from four high-level actions based on context selection. Once a high-level action is selected, the agent transitions to the mitigation level, where it can choose from multiple IDS threshold range options, each defined by specific parameters. High-level actions in all HRL variations do not alter the environment directly, but contribute to shaping low-level states.

4.4 Reward Structures

The reward function handles the reward structures for both the high- and low-level state-action pairs. The environment stores all the thresholds for each variable and inspects if the incoming values are within the specified ranges for the calculation of FPs and FNs. It also keeps track of the detection flags. The high-level scalar reward is calculated based on high-level actions and state variables. On the other hand, the low-level vector reward is calculated based on the following dynamic cyber incident-related factors: damage, cost, FPs, and FNs. The reward function is designed to encourage the agent to limit the instability factors while encouraging long-term stability. The reward function is also divided into two stages to keep a balance between reward shaping and state representation. The total reward signal is derived by aggregating the high-level (R_{HL}) and low-level reward (R_{LL}) (Equation (5)):

$$Reward = R_{HL} + R_{LL}. \tag{5}$$

4.4.1 High-Level Reward. The model assumes the existence of n parameters in the high-level state vector. At timestep t, the value of the current index of the parameter k is denoted by $s_H^k(t)$. At this level, the agent is only looking for the anomaly at the network level. In case of anomaly, the agent will look for further details (e.g., the context). We represent the anomaly at this level as threshold violations. Each high-level threshold η^H has an upper bound u for any given parameter k that can be represented as η_u^{kH}. Similarly, the lower-bound l for a parameter k for the high-level threshold can be denoted as η_l^{kH}. The detection flag $D_k(t)$ and the high-level reward can be calculated as follows:

$$R_{HL} = \begin{cases} 1 & \text{if } s_H^k(t) \leq \eta_u^{kH} \text{ and } s_H^k(t) \geq \eta_l^{kH}, D_k(t) = 1 \\ 0 & otherwise, D_k(t) = 0 \end{cases}. \tag{6}$$

4.4.2 Low-Level Reward. The low-level reward involves several objectives, each calculated as a scalar value. These low-level rewards will form the final reward vector. The various objectives are calculated as follows:

— The cost of action $C(a)$ is based on the activity level and the power cost at a specific time. For example, if the current production activity level is low, and the threshold for the traffic flow range is set above the normal value (even if it is in the acceptable range), this action may not be optimal. Similarly, adjusting the range to a lower threshold (even if within the acceptable

level) during peak production periods might also be sub-optimal. This cost of the action is incorporated into the reward calculation.

—Cost of FP $FP_k(t)$. A FP occurs when the system incorrectly flags normal behavior as suspicious. In our case, it would be when the incoming parameter's kth value is out of the range set of the current threshold for that parameter. The kth value at time t for the lower level state parameter is given by $s_H^k(t)$. The threshold range's lower bound is represented as η_l^{kH} and the upper-bound of the range is represented as η_u^{kH}. Using this, we mathematically represent current FPs $FP_k(t)$ using Equation (7). The *detection* flag $D_k(t)$ is calculated at the higher level using the Equation (6):

$$FP_k(t) = \begin{cases} 1 & \text{if } s_H^k(t) \leq \eta_u^{kH} \text{ and } s_H^k(t) \geq \eta_l^{kH} \text{ and } D_k(t) == 1 \\ 0 & otherwise \end{cases}. \tag{7}$$

—Cost of Missed Detection $FN_k(t)$. A missed detection happens when the system fails to detect an actual suspicious activity. We represent the missed detection or a FN at time t using Equation (8), where η_l^{kH} is the lower bound of the threshold η for the kth parameter and η_u^{kH} is the threshold upper bound for the same parameter. The *detection* flag $D_k(t)$ is calculated at the higher level using Equation (6):

$$FN_k(t) = \begin{cases} 1 & \text{if } (s_H^k(t) < \eta_u^{kH} \text{ or } s_H^k(t) > \eta_l^{kH}) \text{ and } D_k(t) == 0 \\ 0 & otherwise \end{cases}. \tag{8}$$

—Expected Damage $D(s)$. Each sensor reading is indirectly indicative of some expected damage. For example, if the flow of the liquid is too high or too low, there is a chance of corrosion or depletion. We use this value in the low-level reward function calculation. This value is calculated separately for IT and the type of OT.

The low-level reward calculation also involves reward weights, denoted by $\alpha,\ \beta,\ \gamma\ and,\ \delta$. For OT-based threat detection in complex CPS, designing cyber incident detection and response systems that are efficient but also effective becomes a complex task. Considering this, we sought to explore the concept of *weighted variables* which often refer to the importance or influence assigned to different elements within the learning process [22]. The assigned weights can be used to prioritize certain actions, states, or experiences over others, which helps the learning algorithm to focus on more relevant or critical aspects of the environment. To emphasize the OT critical components of the cyber incident detection task, we assigned a higher weight to the OT network variables. This kind of enhancement can be potentially effective in mission-critical CPS. The weighted variable approach can amplify the overall HRL process where different variables might correspond to different levels of abstraction or different aspects of a task.

The final reward calculation of each of these factors is influenced by the weight assigned ($\alpha,\ \beta,\ \gamma,\ and\ \delta$) to it, reflecting its level of importance as follows:

$$R_{LL}(s,a) = [\alpha D(s),\ \beta FP_k,\ \gamma FN_k\ ,\ \delta C(a)]. \tag{9}$$

For a detailed breakdown of the four factors and the calculations of their corresponding objectives, readers are invited to refer to Appendix A. A key advantage of our proposed approach lies in its flexibility: Irrespective of the specific implementation of the low-level goals, the overall model remains robust and adaptable. This hierarchical structure allows the agent to learn reusable skills and strategies. This can ultimately simplify the overall learning process and lead to more robust solutions compared to trying to learn a monolithic policy, which can enhance generalization.

4.5 Option-Critic Implementation

The option-critic HRL framework uses the concept of options (temporally extended actions). It consists of two main components: option-policy and option-critic. Option-policy determines which option to execute, and option-critic then evaluates the value of different available options. We used this model with two different slight variations.

4.5.1 High-Level Model. In the first option-critic architecture, the agent takes a high-level decision to select the context before it moves to the lower level actions of selecting the threshold ranges. The high-level selects an option, and the low-level follows it until the termination condition is met. This approach is different than the standard option-critic architecture where the agent selects a single high-level option and executes low-level actions based on the options policy. The high-level policy π_Ω decides between the contexts IT or OT and selects the corresponding option (setting the threshold ranges). The policy over options π_ω is split into policies for IT or OT based on the implicit high-level states s_t. s_t has both high-level and low-level states and are used implicitly by each level. The termination for the context is evaluated based on the termination function evaluated by the agent after each episode. Once the context is selected, the low level policy π_ω selects one of the threshold ranges-the low-level action a_t. The π_ω can be influenced by the high-level context to prioritize different actions that effectively conditions the agent's behavior. The low-level actions are decided based on the portability calculated by the low-policy network. A single critic network evaluates the Q-values for each combination of high-level context with low-level policies. It also calculates the temporal difference error, which is then used to adjust π_Ω and π_ω. Once the action a_t is executed, the reward r_t is received. R_{HL} is calculated based on the correct context selection, and the low-level reward R_{LL} provides feedback on the success of individual actions in achieving immediate goals. The low-level reward is a combined value from Equation (9). The option terminates when the termination condition is met. In this implementation, we only have time-based termination conditions, β_{IT} and β_{OT}, which act as the switch and wait for the low-level action to be executed. The implemented architecture is shown in Figure 3, where the option-critic agent is represented with dotted line.

In the second variation of this implementation, we used a multi-head option-critic implementation, where each option head corresponds to a different option, allowing the architecture to handle multiple distinctive goals (damage, cost, FP, and FN) at the low-level policy. Instead of a single option head, there are multiple heads for each factor making it easy for the agent to learn separate low-level polices for each potential cyber incident-related factor. The reward for each factor contributes to evaluating the action separately, because each factor in Equation (9) affects their respective option evaluation. In this scenario, each option head operates independently, conditioning the low-level policy for actions related to that factor and giving it more flexibility to optimize each one individually. These option heads operate independently and do not influence each other directly, but they all condition the low-level policy for actions in the environment. There are also multiple-critic networks, which evaluate multiple Q-values, one for each cyber-incident-related factor and one for each context. This independence is also applied to the termination as well. For example, the option head for *cost* will terminate and re-evaluate the option independently of the option head for *damage*. When an option is met, the option head can terminate, independently from the others.

4.5.2 Low-Level Algorithmic Implementation. Following the high-level model, the option-policy part takes the input states. Here, the high and low distinction of the states is implicit, and the framework is presented with the full state (both high and low levels) at the same time. Once the input states are analyzed, the option-critic evaluates each option based on the reward function.

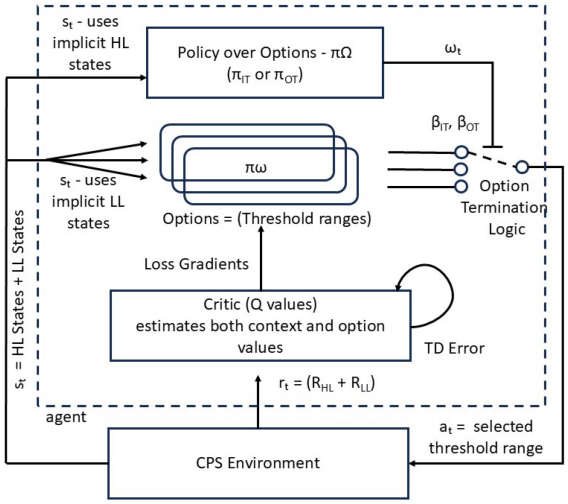

Fig. 3. Adapted option-critic architecture for solving the adaptive IDS threshold problem.

Then, feedback is propagated to the option-policy to choose the best option to execute. The option-critic agent learns via the policy gradient method to select options that lead to higher rewards. In the first implementation, the single-option critic architecture has three networks: one for the option-value network (high-level action), the second for the option termination network, and the third for the intra-option policy (low-level actions). The framework takes in a full state (high and low) as input to its option-value network, then outputs a single fully connected layer that produces a single Q-value for each option. This network has two hidden layers with ReLU activation function. The high-level state is also received by the option-termination network that outputs the termination probabilities for each option. The output layer has a sigmoid activation function, since a decision of whether the option should end or not is required to terminate. If a goal is achieved, the option termination signal is generated.

The termination network also has two hidden layers with ReLU function activation. For the intra-option policy network, we use an nn.ModuleList with a separate network for each option. The input is the low-level state with two fully connected layers having ReLU activations (nn.Linear(64,32)); this layer outputs a soft-max layer to give the probability for each option. The single option-critic architecture computes only one Q-value for each option, covering all factors in a single value. To allow the network to independently learn rewards and Q-values for each factor, we implemented the multi-option critic model, which also has three components: the option-value networks (a separate network for each option), intra-option policy network, and an option-termination network. The framework receives high-level states and low-level states as input, and routes them to the termination network and option-value networks. The termination network outputs a probability for each factor to determine if the option should be terminated or not. This component has two hidden layers and an output layer with soft-max activation. The option termination also takes in the full state (high and low) and outputs a fully connected layer with ReLU activation. There are two fully connected hidden layers followed by a fully connected output layer followed by Sigmoid activation to produce the termination probability for each options. The final component is the option-value networks; it has a single fully connected layer for each option, and produces separate Q-values for each factor.

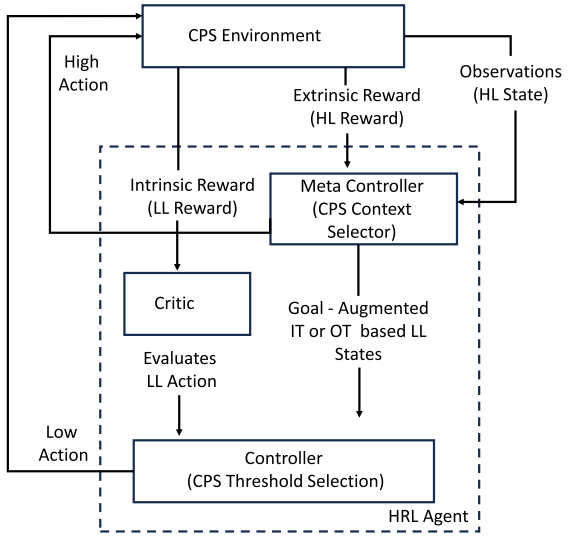

Fig. 4. Adopted h-DQN architecture for solving our HRL problem.

4.6 h-DQN Algorithm

The architecture is based on an explicit hierarchical structure of DQNs consisting of two levels: The high level helps determine an overall strategy or goal, while the low-level policy executes the action selected by the high-level policy. The high-level framework first takes the high-level states as the input and outputs a goal or a task (a context in our scenario). The low-level policy then takes the low-level states based on the selected action by the high-level policy and outputs a low-level action to execute the task. The learning is done at both level simultaneously with the high-level policy learning to select appropriate sub-tasks and the low-level policies learning to execute them effectively.

4.6.1 High-Level Model. Figure 4 shows the h-DQN architecture for the combined implementation that we adopt for this research. The agent is represented with a dotted line. The architecture has two components, a *meta-controller* for high-level policy generation, and a *controller* for low-level policy generation. The meta-controller takes in the observations (high states) as input to the high-level Q-network. It estimates the value function for the sub-goals and outputs the sub-goal or the context for the lower Q-network. This goal is the high-level objective for the low-level controller. Based on the context or the sub-goal the controller takes, the agent estimates the value function for the low actions in the augmented low-level states. A critic evaluates the performance of the controller based on the intrinsic reward it receives from the environment. The meta-controller sub-goal selection is also evaluated based on the high-level extrinsic rewards. This architecture combines both high-level and low-level policies into a single architecture, where the overall architecture takes the high states and executes the low-level tasks based on the best policy. The architecture is proposed to facilitate a hierarchical decision-making structure where a high-level policy sets the stage for a low-level policy to operate. This approach leverages the strengths of HRL and attempts to tackle the complex threat detection and mitigation problem in a CPS by breaking it down into manageable sub-tasks.

4.6.2 Low-Level Algorithmic Implementation. The two DQN networks are implemented with fully connected (dense) layers. The high-level DQN network (selects the sub-goals) consists of a

fully connected layer and takes in high-level states, hence it has the dimension equal to the features in the high-level state. The next layer is the hidden layer, which introduces non-linearity using the ReLU activation function, followed by the second hidden layer also using the ReLU function. The final layer is the output layer, which is fully connected to the previous layer to keep the raw Q-values and uses no activation function. This layer outputs Q-values for each high-level action. Once the action is selected, the state is augmented with more information. The low-level DQN takes in the high low-level state and the high-level action. The input layer dimension for this network then becomes the sum of the high-state features, low-level state features, and the number of the high-level states. The high-level actions are one-hot encoded prior to the low-level input. The combined, high-level, low-level, and the high-actions are fed into a fully connected layer. This DQN also has two hidden layers with ReLU activation. The output layer produces the Q-values for each low-level action. Again, no activation is applied to keep the raw Q-values for decision making. The network is intended to learn complex representations of the input data and reduce the dimensions of the state space. Following the primary testing of the combined signal model, we found that the agent lacked the precision in handling different factors at the low level individually. Thus, we updated the model with separate high- and low-level reward signals. The high- and low-level states are the same as the combined variation, but low-level actions were associated with distinct reward signals for the cost of FP, FNs, the cost of action, and the presence of dynamic damage of the subsystem. This provides more granularity and better learning compared to the first model.

5 Evaluation and Results

This section presents a comprehensive analysis of the experimental evaluation and its corresponding findings. To rigorously assess the effectiveness of the proposed HRL-enabled cyber defense framework, we employ a two-phase evaluation process. In the first phase, we establish a baseline performance using a random agent that selects each possible action with equal probability (Figure 7). This baseline represents a non-adaptive policy, and its performance exhibits significant fluctuation across episodes, with total rewards ranging between 8 and 16. These results suggest the presence of environmental stochasticity and reinforce the ineffectiveness of random action selection in providing consistent or meaningful defense. This baseline serves as a comparative reference for evaluating the performance of all HRL agents. All agents consistently outperform the random baseline, thereby validating both the policy learning process and the optimization capabilities of our proposed framework. To simulate realistic CPS behavior, we employ a Network Data Generator that constructs the environment by generating both high-level and low-level state representations. This component provides comprehensive metrics from both IT and OT domains, and it initializes the environment with default threshold values. The initial threshold bounds, denoted by η_u and η_l, are derived from the normal operating ranges of system metrics. The static baseline analysis evaluates cyber incident detection under fixed environmental conditions. However, this rigid evaluation fails to capture the influence of time-varying operational dynamics, which are effectively addressed by the autonomous HRL-based detection framework. For example, under HRL, parameters such as power rate are dynamically categorized as high, medium, or low, directly influencing cost-related decisions. In contrast, the static method assumes a constant power rate of 0.8. Similarly, production capacity is adaptively adjusted based on temporal context (e.g., time of day), whereas the baseline relies on a fixed value.

In the second phase of evaluation, we use the same environmental configuration and performance metrics to assess the HRL environment across four algorithmic variants: (1) Option-Critic with Single Head, (2) Option-Critic with Multiple Heads, (3) h-DQN with Combined Reward, and (4) h-DQN with Separate Rewards for high-level and low-level controllers. Each HRL agent operates in the generated CPS environment, leveraging dynamic network conditions to estimate and respond to evolving cyber

threats. These threats are represented through both high-level and low-level state observations. The agents autonomously adapt detection thresholds in response to changing operational metrics and network behaviors, enabling timely and context-aware mitigation of cyber risks.

5.1 h-DQN Agent with Combined Reward

In this variation, we employed a combined reward signal, as defined in the proposed architecture, to guide the learning process of the HRL agent. The model was trained over 1,000 episodes, followed by 50 test episodes for evaluation. The agent successfully reduced the cumulative reward to a level notably lower than that of the static baseline, demonstrating improved performance in managing both FPs and FNs. However, during training, the reward signal exhibited significant fluctuations, ranging between 100 and 180, indicating instability in the learning process. This instability is primarily attributed to the non-monotonic nature of the combined reward signal, which posed challenges for the agent in maintaining consistent learning progress. To address this issue, we conducted a detailed analysis of the reward structure and the associated weights corresponding to the four optimization factors targeted by the agent. Various solutions were considered, including reward reshaping, state space refinement, and annealing of reward weights. Among these, we selected the annealing weights approach, which gradually adjusts the contribution of each reward component over time. This method was adopted to mitigate the impact of abrupt dynamic shifts during training and to improve learning stability across episodes.

Following the integration of annealed weights into the reward structure, the training process continued to exhibit fluctuations; however, the average reward range narrowed and stabilized between 120 and 180. The testing phase showed improved stability, with rewards consistently maintained between 130 and 134, as illustrated in Figure 5, indicating overall steady and reliable performance. To further enhance exploration, we reduced the epsilon decay rate, allowing the agent to explore more extensively during early training stages. The epsilon value decayed smoothly from 1.0 to 0.05 over the course of 6,000 episodes, after which it remained constant for the remainder of the training process, as designed. In terms of factor-specific performance, the updated model achieved a noticeable reduction in overall cost compared to earlier configurations. The relatively stable testing rewards, particularly in contrast to the more variable training rewards, suggest that the agent has learned a generalizable policy capable of performing well on unseen data. The minor variations in test reward are likely attributable to residual tuning needs and could potentially be mitigated through additional hyperparameter refinement. Despite initial training challenges, the agent ultimately demonstrated robust performance in detecting cyber threats and effectively mitigating less complex indicators, such as FPs and FNs. This was achieved through its ability to autonomously adapt incident detection thresholds based on evolving environmental conditions, validating the utility of the HRL-based approach.

5.2 h-DQN Agent with Separate High and Low Rewards

Based on insights from the previous model's performance, we revised the reward structure to address observed variability by introducing separate reward signals for the high-level and low-level controllers. This modification enables more precise tuning of the agent's policy at each decision-making layer. While the agent learns from the decoupled high and low rewards, we retained the combined reward solely for comparison and visualization purposes. Specifically, the combined reward is logged for plotting (as shown in the leftmost and rightmost plots of Figure 6), while the learning function operates on the separate components. During training, the high-level reward exhibits a noticeable increase around episode 800, albeit with intermittent fluctuations. In contrast, the low-level reward rises more gradually and consistently over time, indicating that the agent was able to learn the high-level policy earlier and then focus on refining low-level control. The combined

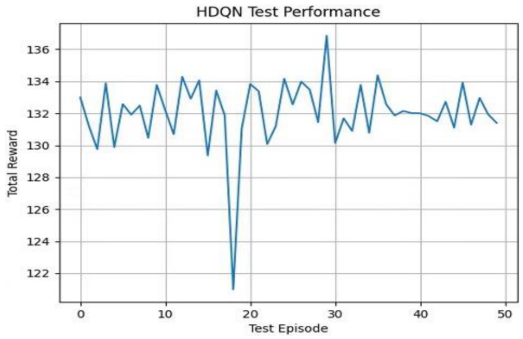

Fig. 5. h-DQN test performance (combined).

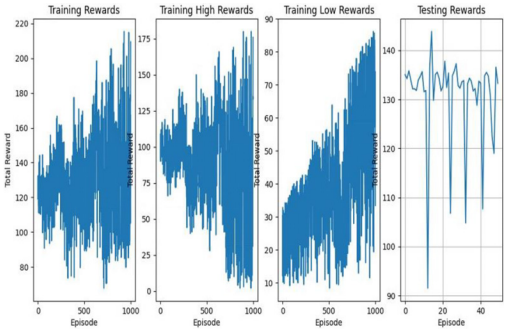

Fig. 6. h-DQN test and train performance (separate).

testing reward demonstrates improved stability compared to the previous model, with values consistently ranging between 130 and 140, suggesting a more robust and generalizable learned policy.

Although there was a slight increase in the damage factor, the agent maintained an overall stable performance. Additionally, the learning rate and epsilon decay followed a smooth and gradual decrease, stabilizing around episode 400—indicative of a well-calibrated tradeoff between exploration and exploitation. Despite experiencing some fluctuations during both training and testing, the agent successfully demonstrated its capability to detect cyber threats and adaptively respond to evolving risks. Notably, it was able to autonomously adjust detection thresholds to mitigate high-impact indicators effectively, thus validating the advantage of the hierarchical approach with a disaggregated reward structure.

5.3 Option-Critic Agent with Single Option

The option-critic model was implemented to evaluate the differences between implicit and explicit task discovery in HRL. While h-DQN represents a classical value-based HRL approach grounded in Q-learning with explicit high- and low-level structures, the option-critic framework utilizes a policy gradient method—specifically, a PPO variant—without an explicit separation between high- and low-level states. In this model, both high-level and low-level state information are presented concurrently to the agent, allowing it to learn a unified policy directly from environmental interactions. A thorough hyperparameter tuning process was conducted to determine an appropriate learning rate and epsilon decay rate that would ensure sufficient exploration while maintaining stable convergence. The agent achieved more reliable performance when using a low epsilon decay rate (0.999) and a modest learning rate (5×10^{-4}). These settings helped reduce instability, but the agent continued to face challenges when attempting to optimize all four evaluation factors simultaneously. In many cases, improvements in one objective occurred at the expense of another, revealing tradeoffs in multi-objective optimization under this configuration.

Notably, the high-level reward signal exhibited sudden and steep fluctuations during training, suggesting that the agent was learning a high-level policy too rapidly—potentially due to inadequately defined termination conditions for high- and low-level options. Revisiting and refining these termination criteria may improve policy stability. Figure 8 displays the trajectory of low-level rewards, which show relatively stable behavior with minimal fluctuations across episodes. This pattern was consistent during testing, further confirming the model's capacity for stable low-level policy learning. Although the overall performance of the option-critic agent was not consistently optimal across all metrics, it effectively demonstrated the ability to detect active cyber threats and autonomously adapt optimal response settings. These results validate the model's potential, with

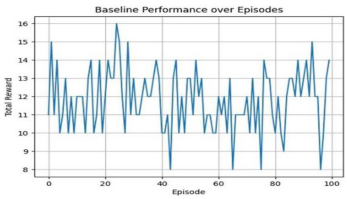

Fig. 7. Baseline performance of random agent.

Fig. 8. Option-critic single head low-level policy training.

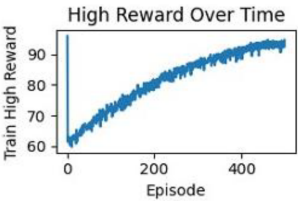

Fig. 9. Option-critic with multi-head high-level reward training.

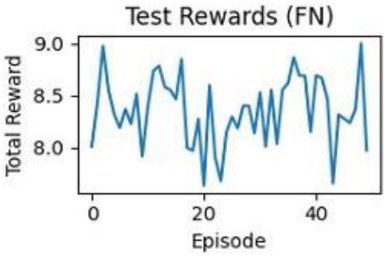

Fig. 10. Option-critic test performance for FN option.

Fig. 11. Option-critic training for the damage option.

room for further enhancement through refined reward shaping, improved termination logic, and targeted policy regularization.

5.4 Option-Critic with Separate Options

This model extends the previously discussed option-critic framework by introducing separate option heads, enabling the agent to distinguish between high-level and low-level policy learning. In the single-option variant, low-level actions were directly influenced by low-level rewards, allowing the agent to perform well in optimizing short-term objectives. However, it struggled with the convergence of high-level rewards, indicating suboptimal long-term policy development. In contrast, the separate-options model exhibited a steady increase in high-level rewards during training, alongside improvements across the four reward components corresponding to distinct cyber defense factors. This upward trajectory suggests that the agent is learning a more effective high-level strategy by gradually optimizing for the cumulative reward—a positive indicator of long-term policy learning (Figure 9). Among all tested models, this variant demonstrated the strongest overall performance in optimizing key indicators for cyber incident detection and mitigation. The damage reward showed a consistent and stable increase throughout training, indicating effective learning of the dynamic threat environment. Although the agent encountered some difficulty optimizing the cost reward, its performance was still superior compared to the other models.

Both FP and FN rewards remained relatively stable, with minor fluctuations, further reinforcing the model's robustness. Figure 10 illustrates the FP reward trend during testing, which the agent maintained within a stable and acceptable range. Similarly, Figure 11 shows the training trajectory for the damage reward, demonstrating the agent's ability to adaptively optimize this critical threat indicator. Figure 12 presents the system behavior under the static method, highlighting persistent cyber incidents, while Figure 13 illustrates the dynamic threat mitigation achieved through the multi-option option-critic agent. The latter clearly shows a reduction in threat presence, validating the model's adaptive response capabilities. Overall, this model outperformed all other

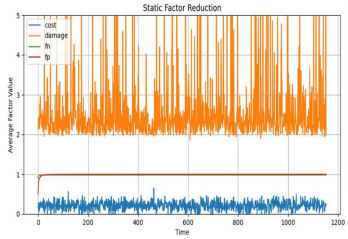

Fig. 12. Static cyber incident presence measure.

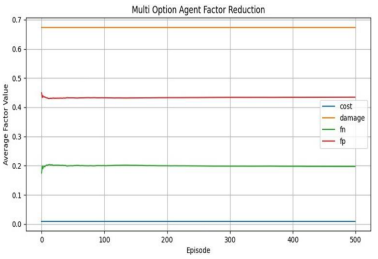

Fig. 13. Option-critic optimized incident indicator reduction.

Table 2. Performance Analysis of the Static Method, h-DQN (Simple), h-DQN (Separate Rewards), Option-Critic (Simple), and Option-Critic (Multi-Head) Based on Dynamic Threat Reduction

Method	Damage	Cost	FPs	FNs
Static	3.1314	0.214069	0.9816	0.997
Option-Critic Multiple Heads	0.6418	0.01	0.1968	0.4345
Option-Critic Simple	0.97226	0.0546	0.19631	0.43650
h-DQN—Separate Rewards	0.8319	0.06333	0.9998	0.9997
h-DQN—Simple Rewards	0.7828	0.0604	0.9998	0.9997

variants, confirming the effectiveness of HRL-based architectures with multi-factor optimization objectives for autonomous cyber incident detection and response. These findings are summarized in Table 2, which demonstrates that all HRL-based models substantially outperformed the static baseline—particularly in minimizing system damage and enhancing adaptive threat response.

6 Conclusion

This article presents a novel AI-enabled solution for adaptive cyber incident detection and response in industrial CPS. We propose an autonomous agent capable of optimizing multiple cyber incident indicators and dynamically adjusting detection thresholds based on real-time threat assessments in industrial CPS environments. To address the challenges posed by large and complex state spaces, we adopt a HRL framework, which decomposes the adaptive thresholding problem into more tractable sub-tasks. Specifically, we explore and compare both value-based (h-DQN) and policy-based (Option-Critic) HRL approaches to highlight the fundamental differences between explicit and implicit hierarchical control. Rather than exhaustively testing every HRL variant—an approach that would be computationally intensive and yield limited additional insight—we focus on evaluating representative architectures that illustrate the key distinctions in learning dynamics and performance. Four HRL agents were implemented: two based on explicit sub-task decomposition (h-DQN) and two based on implicit sub-task discovery (Option-Critic). These agents were evaluated based on reward progression, comparative performance across architectures, and their effectiveness in reducing dynamic instability factors within the CPS. The experimental results demonstrate that the proposed agents can autonomously adapt detection thresholds while maintaining real-time system performance. Central to this adaptability is the reward function design, which directly penalizes dynamic system damage and cost. This incentivizes the agent to learn behavior that preserves system integrity, independent of specific system states or minor environmental variations. The modularity of the HRL framework further supports long-term generalization. As system requirements evolve, the hierarchical structure can be extended or refined by incorporating new

sub-policies or adjusting existing ones. Thus, as long as high-level goals and low-level actions are well-aligned with the underlying risk model, the learned policies remain robust, scalable, and adaptable across a wide range of operational scenarios.

Future research will focus on extending the hierarchical structure of the proposed autonomous cyber threat detection and mitigation framework to enable the agent to reason and act across multiple levels of abstraction (expanded action space), thereby enhancing its decision-making granularity and strategic flexibility. We also plan to explore the integration of multi-agent systems within the low-level action space to facilitate collaborative and coordinated responses to complex, distributed cyber threats. While the current study addresses adaptive risk response, it does not empirically assess adversarial resilience. However, the reward structure—where adversary-induced damage is reflected as a reduction in cumulative reward—offers a promising foundation for modeling adversarial dynamics. This will allow us to evaluate the system's capacity for real-time threat adaptation and proactive defense. To further validate the robustness and scalability of our approach, we intend to evaluate the framework in a real-world CPS environment.

Acknowledgment

The statements made herein are solely the responsibility of the authors.

References

[1] Lucas N. Alegre, Ana L. C. Bazzan, and Bruno C. da Silva. 2021. Minimum-delay adaptation in non-stationary reinforcement learning via online high-confidence change-point detection. In *20th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS '21)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 97–105.

[2] Shamshair Ali, Saif Ur Rehman, Azhar Imran, Ghazif Adeem, Zafar Iqbal, and Ki-Il Kim. 2022. Comparative evaluation of AI-based techniques for zero-day attacks detection. *Electronics* 11, 23 (2022), 3934.

[3] Abdullah Alsaedi, Nour Moustafa, Zahir Tari, Abdun Mahmood, and Adnan Anwar. 2020. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access* 8 (2020), 165130–165150.

[4] Mihai Anca, Mark F. Hansen, and Matthew Studley. 2023. Modular hierarchical reinforcement learning for robotics: Improving scalability and generalizability. In *ICML Workshop on New Frontiers in Learning, Control, and Dynamical Systems*.

[5] Andrew Barto and Richard S. Sutton. 1998. *Reinforcement Learning: An Introduction*. Vol. 1, MIT Press, Cambridge.

[6] Pierre-Luc Bacon, Jean Harb, and Doina Precup. 2017. The option-critic architecture. In *the AAAI Conference on Artificial Intelligence*, Vol. 31.

[7] Taposh Banerjee, Miao Liu, and Jonathan P. How. 2017. Quickest change detection approach to optimal control in Markov decision processes with model changes. In *2017 American Control Conference (ACC)*, 399–405. DOI: https://doi.org/10.23919/ACC.2017.7962986

[8] Andrew G. Barto. 2013. Intrinsic motivation and reinforcement learning. In *Intrinsically Motivated Learning in Natural and Artificial Systems*. G. Baldassarre and M. Mirolli (Eds.), Springer, 17–47.

[9] Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. 1970. A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics* 41, 1 (1970), 164–171.

[10] Marc G. Bellemare, Will Dabney, and Rémi Munos. 2017. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*. PMLR, 449–458.

[11] Dimitri Bertsekas and John N. Tsitsiklis. 1996. *Neuro-Dynamic Programming*. Athena Scientific.

[12] Leyla Bilge and Tudor Dumitraş. 2012. Before we knew it: An empirical study of zero-day attacks in the real world. In *2012 ACM Conference on Computer and Communications Security*, 833–844.

[13] Vivek S. Borkar. 2009. *Stochastic Approximation: A Dynamical Systems Viewpoint*, Vol. 48. Springer.

[14] Chuck Brooks. 2021. Cybersecurity Threats: The Daunting Challenge of Securing the Internet of Things. Retrieved from https://www.forbes.com/sites/chuckbrooks/2021/02/07/cybersecurity-threats-the-daunting-challenge-of-securing-the-internet-of-things

[15] Giuseppe Canonaco, Marcello Restelli, and Manuel Roveri. 2020. Model-free non-stationarity detection and adaptation in reinforcement learning. In *European Conference on Artificial Intelligence*. Retrieved from https://api.semanticscholar.org/CorpusID:221714335

[16] Axel Charpentier, Christopher Neal, Nora Boulahia-Cuppens, Frederic Cuppens, and Reda Yaich. 2023. Real-time defensive strategy selection via deep reinforcement learning. In *18th International Conference on Availability, Reliability and Security (ARES '23)*. ACM, New York, NY, Article 15, 11 pages. DOI: https://doi.org/10.1145/3600160.3600176

[17] Tong Chen, Jiqiang Liu, Yingxiao Xiang, Wenjia Niu, Endong Tong, and Zhen Han. 2019. Adversarial attack and defense in reinforcement learning-from AI security view. *Cybersecurity* 2 (2019), 1–22.

[18] Joseph Clark. 2023. DOD Releases AI Adoption Strategy. Retrieved February 21, 2024 from https://www.defense.gov/News/News-Stories/Article/Article/3578219/dod-releases-ai-adoption-strategy/

[19] Austin Cooper and Sean P. Meyn. 2024. Reinforcement learning design for quickest change detection. arXiv:2403.14109. Retrieved from https://api.semanticscholar.org/CorpusID:268553633

[20] Peter Dayan and Geoffrey E. Hinton. 1992. Feudal reinforcement learning. In *Advances in Neural Information Processing Systems*, Vol. 5.

[21] Thomas G. Dietterich. 2000. Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research* 13 (2000), 227–303.

[22] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. 2021. Challenges of real-world reinforcement learning: Definitions, benchmarks and analysis. *Machine Learning* 110, 9 (2021), 2419–2468.

[23] Mohamed Amine Ferrag, Othmane Friha, Djallel Hamouda, Leandros Maglaras, and Helge Janicke. 2022. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. *IEEE Access* 10 (2022), 40281–40306.

[24] FireEye Technical Report. 2020. Zero-Day Exploitation Increasingly Demonstrates Access to Money, Rather Than Skill—Intelligence for Vulnerability Management. Retrieved February 19, 2024 from https://www.fireeye.com/blog/threat-research/2020/04/zero-day-exploitation-demonstrates-access-to-money-not-skill.html

[25] M. Fortunato, M. G. Azar, B. Piot, J. Menick, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, et al. 2017. Noisy networks for exploration. arXiv:1706.10295. Retrieved from https://arxiv.org/abs/1706.10295

[26] Amin Ghafouri, Waseem Abbas, Aron Laszka, Yevgeniy Vorobeychik, and Xenofon Koutsoukos. 2016. Optimal thresholds for anomaly-based intrusion detection in dynamical environments. In *7th International Conference on Decision and Game Theory for Security (GameSec '16)*. Springer, 415–434.

[27] Freepik Gravisio and Witdhawaty. 2024. Icons Created by Flaticon. Retrieved February 22, 2024 from https://www.flaticon.com/

[28] Abhishek Gupta, Vikash Kumar, Corey Lynch, Sergey Levine, and Karol Hausman. 2019. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. arXiv:1910.11956. Retrieved from https://arxiv.org/abs/1910.11956

[29] Bernhard Hengst. 2011. Hierarchical reinforcement learning. In *Encyclopedia of Machine Learning*. C. Sammut and G. I. Webb (Eds.), Springer, 495–502.

[30] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, and D. Silver. 2017. Rainbow: Combining improvements in deep reinforcement learning. arXiv:1710.02298. Retrieved from https://arxiv.org/abs/1710.02298

[31] Wenning Hu, Hongbin Wang, Ming He, and Nianbin Wang. 2023. Uncertainty-aware hierarchical reinforcement learning for long-horizon tasks. *Applied Intelligence* 53, 23 (2023), 28555–28569.

[32] Linan Huang and Quanyan Zhu. 2019. Adaptive strategic cyber defense for advanced persistent threats in critical infrastructure networks. *SIGMETRICS Performance Evaluation Review* 46, 2 (Jan. 2019), 52–56. DOI: https://doi.org/10.1145/3305218.3305239

[33] Yunhan Huang, Linan Huang, and Quanyan Zhu. 2022. Reinforcement learning for feedback-enabled cyber resilience. *Annual Reviews in Control* 53 (2022), 273–295.

[34] Matthias Hutsebaut-Buysse, Kevin Mets, and Steven Latré. 2022. Hierarchical reinforcement learning: A survey and open research challenges. *Machine Learning and Knowledge Extraction* 4, 1 (2022), 172–221.

[35] William Koch, Renato Mancuso, Richard West, and Azer Bestavros. 2019. Reinforcement learning for UAV attitude control. *ACM Transactions on Cyber-Physical Systems* 3, 2 (2019), 1–21.

[36] Singamaneni Krishnapriya and Sukhvinder Singh. 2024. A comprehensive survey on advanced persistent threat (APT) detection techniques. *Computers, Materials & Continua* 80, 2 (2024), 2675–2719.

[37] Tejas D. Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. 2016. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. In *Advances in Neural Information Processing Systems*, Vol. 29, 2016.

[38] Aron Laszka, Waseem Abbas, S. Shankar Sastry, Yevgeniy Vorobeychik, and Xenofon Koutsoukos. 2016. Optimal thresholds for intrusion detection systems. In *Symposium and Bootcamp on the Science of Security*, 72–81.

[39] Andrew Levy, George Konidaris, Robert Platt, and Kate Saenko. 2017. Learning multi-level hierarchies with hindsight. arXiv:1712.00948. Retrieved from https://arxiv.org/abs/1712.00948

[40] Yingying Li and Na Li. 2019. Online learning for Markov decision processes in nonstationary environments: A dynamic regret analysis. In *2019 American Control Conference (ACC)*, 1232–1237. DOI: https://doi.org/10.23919/ACC.2019.8815000
[41] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing Atari with deep reinforcement learning. arXiv:1312.5602. Retrieved from https://arxiv.org/abs/1312.5602
[42] Ofir Nachum, Shixiang Shane Gu, Honglak Lee, and Sergey Levine. 2018. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems*, Vol. 31, 2018.
[43] Thanh Thi Nguyen and Vijay Janapa Reddi. 2021. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2021), 3779–3795.
[44] Sindhu Padakandla. 2021. A survey of reinforcement learning algorithms for dynamically varying environments. *ACM Computing Surveys* 54, 6 (2021), 1–25.
[45] Ronald Parr and Stuart Russell. 1997. Reinforcement learning with hierarchies of machines. In *Advances in Neural Information Processing Systems*, Vol. 10.
[46] Shubham Pateria, Budhitama Subagdja, Ah-Hwee Tan, and Chai Quek. 2021. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys* 54, 5 (2021), 1–35.
[47] Antonio Paya, Sergio Arroni, Vicente García-Díaz, and Alberto Gómez. 2024. Apollon: A robust defense system against adversarial machine learning attacks in intrusion detection systems. *Computers & Security* 136 (2024), 103546.
[48] Yaman Roumani. 2021. Patching zero-day vulnerabilities: An empirical analysis. *Journal of Cybersecurity* 7, 1 (2021), tyab023.
[49] Nitasha Sahani, Ruoxi Zhu, Jin-Hee Cho, and Chen-Ching Liu. 2023. Machine learning-based intrusion detection for smart grid computing: A survey. *ACM Transactions on Cyber-Physical Systems* 7, 2 (2023), 1–31.
[50] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. 2015. Prioritized experience replay. arXiv:1511.05952. Retrieved from https://arxiv.org/abs/1511.05952
[51] Mohit Sewak, Sanjay K. Sahay, and Hemant Rathore. 2021. Deep reinforcement learning for cybersecurity threat detection and protection: A review. In *International Conference on Secure Knowledge Management in Artificial Intelligence Era*. Springer, 51–72.
[52] Mohit Sewak, Sanjay K. Sahay, and Hemant Rathore. 2023. Deep reinforcement learning in the advanced cybersecurity threat detection and protection. *Information Systems Frontiers* 25, 2 (2023), 589–611.
[53] Richard S. Sutton, Doina Precup, and Satinder Singh. 1999. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112, 1–2 (1999), 181–211.
[54] Csaba Szepesvari, Richard S. Sutton, Joseph Modayil, Shalabh Bhatnagar, Doina Precup, and David Silver. 2014. Universal option models. In *Advances in Neural Information Processing Systems*, Vol. 27.
[55] Chen Tessler, Shahar Givony, Tom Zahavy, Daniel Mankowitz, and Shie Mannor. 2017. A deep hierarchical approach to lifelong learning in Minecraft. In *the AAAI Conference on Artificial Intelligence*, Vol. 31.
[56] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 5026–5033.
[57] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. 2016. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning*. PMLR, 1995–2003.
[58] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain. 2021. WUSTL-IIOT-2021 Dataset for IIoT Cybersecurity Research. Washington University in St. Louis. Retrieved from http://www.cse.wustl.edu/jain/iiot2/index.html

A Appendix

We present here the low-level reward calculations explained in Section 4.4.

Cost of Action. The cost is modeled as a function of production capacity (pc) and the dynamic power rate (pr) at that time. We want the agent to stay in the desired threshold ranges while keeping the cost low, keeping in context the activity level. For example, the production capacity is not at its peak in the morning and the power rates might be low. Whereas the production capacity might not be at its peak in the evening either, but the power rates might be high, hence the cost will be higher in the evening but lower in the morning. The cost is calculated as follows:

—pc: The production capacity of the unit at any given time.
—pr: The real-time power rates of the unit at that time.
—t: timestamp (numerical value).

—η_l^{kL}: lower threshold bound for the kth parameter in the lower-level state vector $S_L L$ (numerical value).
—η_u^{kL}: upper threshold bound for the kth parameter in the lower-level state vector $S_L L$ (numerical value).
—$s_L^k(t)$: The kth parameter value of the lower-level state at time t.

We first normalize each incoming value using Equation (A1) then calculate the cost of the action $c(a)$ based on the weights assigned to the variables pr and pc. We call these weights the rush weight (w_{rush}) and the power weight (w_{high}). We calculate the cost of the action based on the dynamic power consumption and production rate using Equation (A2). This cost function assigns a lower cost closer to the middle threshold range and a penalty that increases as the value deviates further from the desired range (average threshold):

$$\text{scalar}(s_L^k(t)) = \frac{\left| s_L^k(t) - \frac{\eta_l^{kL} + \eta_u^{kL}}{2} \right|}{\frac{\eta_u^{kL} - \eta_l^{kL}}{2}}, \tag{A1}$$

$$C(a) = w_{rush}(pc) * w_{high}(pr) * scalar(s_L^k(t)). \tag{A2}$$

Expected Damage. In our experiment, we calculate the expected damage using preliminary equations from the manufacturing domain based on fundamental readings. It is essential to note that the proposed methodology will remain unaffected when incorporating more advanced equations and a broader range of readings during future scalability enhancements. To calculate the expected damage to the system we use three different values: the changes in temperature of the chemical chamber, the power consumption of the overall network, and changes in the flow of the liquid in the pump. The high power consumption can cause the components to throttle and can affect their performance. Similarly, the changes in the temperature are directly related to the chemical process disruption. Moreover, the pressure in the pump can cause vibrations and instability of the equipment and may cause damage.

We calculate the network-based (IT-based) expected damage using the power consumption percentage and the Modbus data rate. We calculate the **Data Transmission Efficiency (DTE)** as per Equation (A3). A high power consumption with low data transmission indicates network interference or insufficient hardware. We calculate the damage in the chemical chamber's changes in temperature using the Arrhenius Equation (A4), where **k** is the rate constant, **A** is the pre-exponential factor (frequency factor), **Ea** is the activation energy (J/mol), **R** is the universal gas constant (8.314 J/mol·K), and **T** is the absolute temperature (K). This equation helps us understand how changes in temperature can affect the rate of a chemical reaction and potentially disrupt the process in a chemical chamber or a reactor. We calculate the expected damage using the Darcy-Weisbach Equation (A5), where **P** is the pressure drop, **f** is the friction factor, **L** is the pipe length, $\boldsymbol{\rho}$ is the fluid density, **Q** is the volumetric flow rate, and **D** is the pipe diameter. The Darcy-Weisbach equation is accurate and universally applicable to calculate and compare the pressure drop:

$$DTE = \frac{Modbus\ rate}{power\ consumption}, \tag{A3}$$

$$k = A * e^{(-Ea/RT)}, \tag{A4}$$

$$P = \left(\frac{f \cdot L \cdot \rho \cdot Q^2}{\pi^2 \cdot D^5} \right). \tag{A5}$$

Received 15 November 2024; revised 1 May 2025; accepted 17 July 2025